

# Análisis de Patrones y Arquetipos

SmartLogix — Evaluación Parcial N°2 · DSY1106 Desarrollo Fullstack III

Mayo 2026

## Contents

|                                                                           |          |
|---------------------------------------------------------------------------|----------|
| <b>1. Resumen</b>                                                         | <b>2</b> |
| <b>2. Patrones arquitectónicos</b>                                        | <b>2</b> |
| 2.1 Microservicios + Database per Service . . . . .                       | 2        |
| 2.2 API Gateway + Ingress separados . . . . .                             | 2        |
| 2.3 Backend For Frontend (BFF) . . . . .                                  | 3        |
| 2.4 Schema-first migrations (Flyway) . . . . .                            | 3        |
| 2.5 Health Probes (Spring Boot Actuator) . . . . .                        | 3        |
| <b>3. Patrones de diseño</b>                                              | <b>3</b> |
| 3.1 Repository Pattern (Data layer) . . . . .                             | 4        |
| 3.2 Service Layer (Business layer) . . . . .                              | 4        |
| 3.3 DTO Pattern (records con <code>from(Entity)</code> ) . . . . .        | 4        |
| 3.4 State Machine . . . . .                                               | 4        |
| 3.5 Optimistic Locking . . . . .                                          | 4        |
| 3.6 Aggregate Root . . . . .                                              | 5        |
| 3.7 Saga simplificada / Composite Service . . . . .                       | 5        |
| 3.8 Circuit-Breaker-lite (AbortController + tolerancia parcial) . . . . . | 5        |
| 3.9 RFC 7807 Problem Detail (errores) . . . . .                           | 5        |
| 3.10 Schema Validation (zod) en BFF . . . . .                             | 6        |
| <b>4. Cómo se cumplen los indicadores 1 y 2 de la rúbrica</b>             | <b>6</b> |
| <b>5. Sobre el “arquetipo”: Gradle como template reutilizable</b>         | <b>6</b> |
| 5.1 ¿Por qué no Maven? . . . . .                                          | 6        |
| 5.2 El “arquetipo” en este proyecto . . . . .                             | 6        |
| 5.3 Si la rúbrica exige Maven literal . . . . .                           | 7        |
| <b>6. Buenas prácticas de desarrollo (ítem 4 de la rúbrica)</b>           | <b>7</b> |
| <b>7. Conclusión</b>                                                      | <b>8</b> |

# 1. Resumen

Este documento describe los **patrones de diseño** y **patrones arquitectónicos** aplicados en SmartLogix, una plataforma logística para PYMEs eCommerce, y justifica por qué cada uno fue elegido en función del problema concreto que resuelve. Incluye además la discusión sobre el uso de **arquetipos** (Maven vs. Gradle como template reutilizable).

El stack final es: **React 19 + Vite** (frontend), **KrakenD** (API Gateway) detrás de **Traefik** (Ingress), **Node.js + Express** (BFF) y **3 microservicios Spring Boot 4 / Java 25** con **PostgreSQL DB-per-Service**, todo orquestado por Docker Compose.

---

## 2. Patrones arquitectónicos

### 2.1 Microservicios + Database per Service

**Problema:** el monolito de logística mezclaba pedidos, inventario y envíos en un solo dominio, generando acoplamiento de equipos, releases lentos y schema compartido difícil de versionar.

**Solución:** tres microservicios independientes (**ms-pedido**, **ms-inventario**, **ms-envio**), cada uno con su propia base PostgreSQL en una red Docker **privada** (**internal: true**). No hay FKs cruzadas: los IDs de cliente, producto o pedido son identificadores **lógicos** que cada servicio interpreta a través de su contrato REST.

**Beneficio:**

- Cada equipo de dominio evoluciona su schema con **Flyway** sin coordinar con los demás.
- Una caída de **ms-envio** no derriba **ms-inventario**.
- Cada servicio se escala horizontalmente de forma independiente (por ejemplo, **ms-inventario** durante peaks de checkout).

**Costo asumido:** consistencia eventual entre servicios. Se mitiga con la saga de checkout descrita en §3.7.

### 2.2 API Gateway + Ingress separados

**Problema:** si el frontend conoce las URLs internas de cada MS, mover servicios o agregar autenticación obliga a redeployar el frontend.

**Solución de dos capas:**

| Capa    | Componente           | Responsabilidad                                                                                                                                             |
|---------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Edge    | <b>Traefik v3.5</b>  | Ingress: TLS termination (preparado), routing por host ( <b>app.*</b> , <b>api.*</b> , <b>bff.*</b> ), middlewares de seguridad (headers, rate-limit, CORS) |
| Gateway | <b>KrakenD v2.10</b> | Routing fino <b>/api/*</b> → BFF, validación JWT (preparada), aggregation/transformación si fuera necesario                                                 |

**Por qué dos capas:** Traefik se encarga de lo que es **cross-cutting de toda la plataforma** (TLS, ingress, frontend estático), mientras que KrakenD se especializa en la rama de tráfico `api.smartlogix.localhost` con políticas declarativas (rate limit por endpoint, JWT validation, CORS específico de API). Esta separación es el patrón estándar en Kubernetes (Ingress + API Gateway), y dejarla así en Docker Compose facilita la migración a K8s después.

## 2.3 Backend For Frontend (BFF)

**Problema:** el frontend necesita datos compuestos de varios MS por pantalla (ej. “*mostrar pedido con sus envíos y disponibilidad de stock*”). Si la SPA hiciera N llamadas en paralelo, la lógica de agregación se duplicaría entre frontend y otros consumidores futuros.

**Solución:** una capa Node.js + Express + zod **dedicada al frontend** que:

1. **Compone** respuestas de varios MS en endpoints como `GET /pedidos/:id/full` o `GET /dashboard`.
2. **Orquesta** flujos transaccionales como `POST /checkout` (crear pedido → reservar stock → crear envío) con **rollback best-effort** ante fallos parciales — ver §3.7.
3. **Hace passthrough** para CRUD simple (`/inventario/*`, `/pedidos/*`, `/envios/*`) con `http-proxy-middleware`.

**Beneficio:** el frontend ve **una API optimizada** para sus pantallas, y la lógica de agregación vive en el servidor (más cerca de los datos, con menor latencia y mejor caché potencial).

## 2.4 Schema-first migrations (Flyway)

**Problema:** con tres bases distintas y `ddl-auto=update` de Hibernate, los schemas divergen entre entornos y nadie sabe cuál es la versión real.

**Solución:** cada MS tiene `V1__init_schema.sql` versionado bajo `src/main/resources/db/migration/`. Hibernate corre en `ddl-auto=validate` — si el código contradice el schema, falla en arranque. Flyway es la **fuentes de verdad**, Hibernate sólo valida.

## 2.5 Health Probes (Spring Boot Actuator)

**Problema:** Docker Compose decide arrancar el BFF cuando los MS están *iniciando* (puerto abierto) pero todavía no son funcionales — Spring Boot tarda 20–30 s en hacer el bootstrap completo. Resultado: el BFF recibe 502 las primeras peticiones.

**Solución:** cada MS expone `/actuator/health` con probes `liveness/readiness`. El `HEALTHCHECK` del Dockerfile valida UP cada 30 s, y el `depends_on: condition: service_healthy` del compose encadena el arranque:

```
db-{inv,ped,env} → ms-{inv,ped,env} → bff → apigateway
```

KrakenD no se levanta hasta que el BFF responde `200 /health`. Los MS no aceptan tráfico hasta que su DB responde `pg_isready` y Spring Boot reporta UP.

---

## 3. Patrones de diseño

Más de tres patrones distintos están aplicados en el código. A continuación se describen los más relevantes, agrupados por capa.

### 3.1 Repository Pattern (Data layer)

**Dónde:** `cl.smartlogix.<dominio>.repository.*Repository` en los 3 MS.

**Implementación:** interfaces que extienden `JpaRepository<Entity, ID>` (Spring Data JPA). Spring genera la implementación en runtime — el código no tiene SQL manual ni boilerplate de `EntityManager`.

**Justificación:** encapsula el acceso a datos detrás de una interfaz orientada al dominio (`OrderRepository.findByCodigo(...)` en lugar de `EntityManager.createQuery(...)`). El service layer no sabe si los datos vienen de JPA, JDBC o de un mock — facilita testing (ver §6).

### 3.2 Service Layer (Business layer)

**Dónde:** `OrderService` (interfaz) + `OrderServiceImpl` (implementación), análogos en los 3 MS.

**Justificación:** separa **qué hace el negocio** (interfaz) de **cómo lo hace** (impl). Toda la lógica de máquina de estados, cálculo de totales con IVA, validaciones cross-tabla, etc., vive aquí, no en los controladores. Las transacciones se anotan a este nivel (`@Transactional`).

### 3.3 DTO Pattern (records con `from(Entity)`)

**Dónde:** `cl.smartlogix.<dominio>.dto.*Request|*Response` — Java records.

**Justificación:** las entidades JPA tienen relaciones lazy, `@Version`, ciclos potenciales y campos internos (`created_at`, etc.). Exponerlas en JSON filtra info y causa errores de serialización. Los DTOs son inmutables (records), validables con Bean Validation y serializables sin sobresaltos.

### 3.4 State Machine

**Dónde:**

- `ms-pedido`: estados `PENDIENTE` → `APROBADO` → `EN_PREPARACION` → `ENVIADO` → `ENTREGADO` (más `RECHAZADO`, `CANCELADO`)
- `ms-envio`: estados `CREADO` → `ASIGNADO` → `EN_RUTA` → `ENTREGADO` con `INCIDENCIA` como rama lateral reintentable

**Implementación:** un `Map<EstadoX, Set<EstadoX>>` `transicionesPermitidas` declarado en el `ServiceImpl`. Cualquier `PATCH /<id>/estado` valida la transición contra este mapa; si es ilegal, se lanza `IllegalStateException` y `GlobalExceptionHandler` la convierte en HTTP 409 `application/problem+json`.

**Justificación:** las reglas del negocio sobre transiciones son **datos** (un grafo), no código procedural disperso. Si un nuevo estado se agrega, se modifica el mapa y nada más.

### 3.5 Optimistic Locking

**Dónde:** campo `@Version private Long version` en `Stock` (`ms-inventario`).

**Justificación:** bajo concurrencia, dos peticiones simultáneas de `POST /stock/salida` sobre el mismo (producto, bodega) corrompen la cantidad disponible. Pesimistic locking (`SELECT ... FOR UPDATE`) bloquea a otros lectores y limita throughput. Con `@Version`, Hibernate incluye `WHERE version = ?` en el `UPDATE` — el segundo escritor recibe `OptimisticLockingFailureException`, que se mapea a HTTP 409 **Conflicto de concurrencia**. El cliente reintenta. Throughput alto, consistencia preservada.

### 3.6 Aggregate Root

Dónde:

- Pedido agrupa `pedido_items` + `pedido_historial`
- Stock agrupa `movimientos_stock`
- Envío agrupa `envio_seguimiento`

**Justificación:** las modificaciones a las hijas se hacen siempre vía la raíz, dentro de la misma transacción. Por ejemplo, cambiar el estado de un envío inserta el `EnvioSeguimiento` correspondiente atómicamente. Esto garantiza el invariante “*no hay envío en estado X sin su entrada de tracking*”.

### 3.7 Saga simplificada / Composite Service

Dónde: `bff/src/services/checkoutService.js`.

Flujo:

1. `POST /pedidos` → ms-pedido crea pedido en PENDIENTE
2. Para cada item: `POST /stock/reservar` → ms-inventario reserva
3. `POST /envios` → ms-envio crea envío en CREADO
4. Si paso 2 falla en algún item: rollback de las reservas anteriores (best-effort)
5. Si paso 3 falla: rollback de la reserva completa

**Justificación:** sin distributed transactions disponibles entre MS, una saga coordinada por el BFF es la forma estándar de garantizar consistencia eventual. Los rollbacks son **compensaciones** (lógicos), no aborts de DB. Si un rollback falla, se logea y se requiere intervención manual — explicitamos esto con el sufijo *best-effort*.

### 3.8 Circuit-Breaker-lite (AbortController + tolerancia parcial)

Dónde: `bff/src/clients/httpClient.js` y `bff/src/services/dashboardService.js`.

Implementación:

- Cada `fetch()` a un MS va envuelto en `AbortController` con `HTTP_TIMEOUT_MS=5000`. Si el MS no responde en 5 s, se lanza `UpstreamError` con status 504.
- `dashboardService` y `pedidoComposerService` ejecutan llamadas a varios MS en paralelo con `Promise.all`, pero capturan errores **por MS** con `.catch()`. Si `ms-envio` falla, el dashboard devuelve datos de pedidos + inventario y deja `enviosEnRuta` vacío.

**Justificación:** un circuit breaker real (Resilience4j, Hystrix) tendría estado (closed → open → half-open) y métricas. Para un BFF stateless y un MVP, el patrón “timeout + degradación parcial” cubre el 80% del valor sin la complejidad operacional.

### 3.9 RFC 7807 Problem Detail (errores)

Dónde: `@RestControllerAdvice` `GlobalExceptionHandler` en cada MS; `bff/src/middleware/errorHandler.js` en el BFF.

**Justificación:** errores como `{"error": "...", "code": 400}` son ad-hoc y cada servicio inventa el suyo. RFC 7807 estandariza el `application/problem+json` con `type`, `title`, `status`, `detail`, `instance` — el frontend tiene **un solo parser de errores** para toda la API.

### 3.10 Schema Validation (zod) en BFF

**Dónde:** `bff/src/schemas/*.js` y `middleware/validate.js`.

**Justificación:** validar la entrada **antes** de tocar los MS evita peticiones desperdiciadas. Si el `idCliente` no es string o falta `items`, zod lo rechaza con 400 antes de salir del BFF. Las entidades del MS, además, tienen Bean Validation propio — defensa en profundidad.

---

## 4. Cómo se cumplen los indicadores 1 y 2 de la rúbrica

**Ítem 1 (10%):** “Implementa al menos 3 patrones de diseño en los componentes frontend y backend, asegurando que los componentes sean eficientes y mantenibles, y justifica la selección de los patrones según el problema que resuelven”.

Diez patrones aplicados, no tres. Cada uno con problema concreto y justificación arriba.

**Ítem 2 (10%):** “Utiliza arquetipos y patrones arquitectónicos adecuados para la construcción del backend (BFF y microservicios)”.

Cinco patrones arquitectónicos aplicados (Microservicios + DB-per-Service, API Gateway + Ingress, BFF, Schema-first migrations, Health Probes), todos con motivación de escalabilidad y eficiencia. La sección §5 cubre el tema “arquetipos” en sentido estricto.

---

## 5. Sobre el “arquetipo”: Gradle como template reutilizable

La rúbrica EV2 menciona explícitamente “**arquetipos Maven**”. Aquí se asumió una desviación deliberada: los tres microservicios fueron generados con **Spring Initializr** (`start.spring.io`) en formato **Gradle 9**.

### 5.1 ¿Por qué no Maven?

1. **Spring Initializr es el arquetipo oficial.** Es la herramienta recomendada por VMware/Broadcom para nuevos proyectos Spring Boot 4, sustituye al viejo `mvn archetype:generate` con un catálogo siempre actualizado, UI y CLI. Por defecto emite **Gradle**.
2. **Performance.** En este monorepo, `./gradlew bootJar` toma ~30 s por MS gracias a incremental build + build cache + paralelización. El equivalente Maven sería 60–90 s.
3. **DSL declarativo más legible.** Los `build.gradle` de los 3 MS son **idénticos** en estructura — el mismo set de plugins (`java`, `jacoco`, `org.springframework.boot`, `io.spring.dependency-management`), las mismas dependencias core, la misma config de JaCoCo. Esto es exactamente lo que un arquetipo busca: **garantizar consistencia entre componentes**. Crear un MS nuevo es literalmente `cp -r ms-pedido ms-nuevo && rename`.
4. **Soporte multi-módulo nativo** con `settings.gradle include`, sin parent POMs.
5. **Tooling moderno.** Spring Framework 7, Spring Boot 4, Kotlin, Android — el ecosistema Spring/JVM contemporáneo usa Gradle internamente.

### 5.2 El “arquetipo” en este proyecto

El template reutilizable existe en la **estructura idéntica** de los 3 MS:

```

ms-<dominio>/
  build.gradle           # plantilla con plugins + deps + JaCoCo
  settings.gradle        # rootProject.name = '<dominio>'
  Dockerfile             # multi-stage build identico
src/main/
  java/cl/smartlogix/<dominio>/
    <D>Application.java    # main
  controller/
    <Recurso>Controller.java
    GlobalExceptionHandler.java
  service/<X>Service{,Impl}.java
  repository/<X>Repository.java
  dto/<X>{Request,Response}.java
  model/<X>.java
  resources/
    application.properties
    db/migration/V1__init_schema.sql
src/test/java/cl/smartlogix/<dominio>/...

```

Generar un MS nuevo es un proceso de **3 pasos**:

```

cp -r backend/microservices/ms-pedido backend/microservices/ms-nuevo
# 1. Renombrar settings.gradle: rootProject.name = 'nuevo'
# 2. Renombrar package: cl.smartlogix.pedido -> cl.smartlogix.nuevo
# 3. Editar application.properties: spring.application.name=nuevo

```

### 5.3 Si la rúbrica exige Maven literal

Plan B (no aplicado, documentado para defensa):

1. Generar un arquetipo Maven desde un MS una vez convertido:

```

cd backend/microservices/ms-pedido
mvn archetype:create-from-project
# genera target/generated-sources/archetype/
mvn install # arquetipo instalado en ~/.m2/repository
# uso:
mvn archetype:generate -DarchetypeArtifactId=ms-pedido-archetype

```

2. Convertir un MS a Maven ejecutando gradle init --type pom, o reescribiendo el pom.xml a mano (las dependencias son las mismas, solo cambia la sintaxis).

Cualquiera de las dos opciones es trabajo de **1-2 horas** sobre el código ya estabilizado.

## 6. Buenas prácticas de desarrollo (ítem 4 de la rúbrica)

- Tests unitarios con Mockito + JUnit 5 en los 3 MS. Cobertura actual (INSTRUCTION, JaCoCo 0.8.14):
  - ms-pedido: ~57 %
  - ms-inventario: ~47 %

- `ms-envio`: ~41 %
  - Excluye DTOs (records), entidades JPA (`model/`) y clase `*Application` — no aportan lógica que testear.
  - **Bean Validation** (`@NotNull`, `@Positive`, `@Size`) en todos los DTOs de entrada.
  - **@Transactional** en service layer para garantizar atomicidad.
  - **Inmutabilidad** en DTOs (records).
  - **Logging estructurado** (Spring Boot por defecto + `morgan` en BFF).
  - **Healthchecks reales** vía Actuator + `depends_on: service_healthy` para arranque ordenado.
  - **Migraciones versionadas** (Flyway) — schema reproducible en cualquier entorno.
- 

## 7. Conclusión

SmartLogix aplica deliberadamente **un patrón arquitectónico por problema** (Microservicios para acoplamiento, BFF para agregación, API Gateway + Ingress para cross-cutting, DB-per-Service para autonomía) y **un patrón de diseño por necesidad técnica concreta** (State Machine para reglas de negocio sobre estados, Optimistic Locking para concurrencia en stock, Saga para consistencia eventual, RFC 7807 para errores estandarizados).

La decisión de usar Gradle en lugar de Maven está justificada técnicamente y el rol de “arquetipo” (template reutilizable) se cumple por la **estructura idéntica de los 3 MS** y su `build.gradle` plantilla. Si la rúbrica exige Maven en sentido estricto, el plan B descrito en §5.3 es ejecutable en 1–2 horas.